GStar Assignment 2: Approach and Key Learnings

Overview

In this assignment, I built a sequence of forward attention implementations that starts with unnormalized linear attention and ends with chunked Kimi Delta Attention (KDA). The main idea is that linear attention can summarize previous key/value pairs in a fixed-size state instead of storing the complete token history. DeltaNet then changes the additive write into a correction, and gating adds controlled forgetting over time.

I first implemented each recurrence directly, then converted it into a chunked form. The chunked kernels separate the sequential state update across chunks from output work that can run independently for different chunks, batch/head pairs, and value tiles.

Implementation Approach

Linear and gated linear attention (Problems 1–4)

For linear attention, I used the recurrent state

$$S_t = S_{t-1} + k_t v_t^\top, \qquad o_t = q_t^\top S_t.$$

The current key/value pair is written before the query reads the state, so causal attention includes the diagonal. In the chunked implementation, each query reads the state entering its chunk and combines that with an inclusive lower-triangular product over tokens inside the chunk. Reading the state after updating the entire chunk would leak future tokens.

The state pass is sequential over chunks because the exit state of one chunk is the entry state of the next. The output pass is more parallel: separate programs own different chunk, batch/head, and value-channel tiles. Key-channel tiles are a reduction dimension within each output program.

Problem 4 added one log-decay gate per key channel. I computed an FP32 prefix sum of the gates inside each chunk. These prefixes let the state kernel move every write to the chunk boundary and let the output kernel decay the entry-state read by the correct amount.

DeltaNet and the WY representation (Problems 5–7)

DeltaNet predicts the value stored at a key and writes only the residual:

$$u_t = \beta_t\left(v_t - k_t^\top S\right), \qquad S \leftarrow S + k_t u_t^\top.$$

With a normalized key and `beta=1`, reading the corrected state at the same key returns the new value exactly. An additive write would add the new value on top of the old prediction instead of replacing that association.

For the chunked version, I built the strictly lower-triangular key Gram matrix and computed the WY representation using `A = (I + L)^-1`. I implemented the inverse with forward substitution inside small diagonal blocks, followed by pairwise block merges. The resulting `W` and `U` factors produce pseudo-values that account for all earlier corrections in the chunk.

Problem 7 added a scalar gate per token. The carried entry state is decayed exactly once per chunk, before adding the exit-decayed corrected writes. Applying this decay inside the tiled update loop would incorrectly apply the same chunk decay multiple times.

Stable channel-wise KDA (Problems 8–9)

KDA uses a different gate for every key channel. Its local score contains a channel-dependent exponential factor. On diagonal subtiles, I computed the gated score directly and masked future entries. On off-diagonal subtiles, I factored the decay around a shared anchor. Because log gates are non-positive, both anchored exponent arguments are non-positive, which prevents overflow while preserving the original relative decay.

The final P9 implementation composes earlier work: the P4 prefix scan, the P6 triangular inverse, and the P8 gated-score builder.

Precision and GPU Resource Use

I kept recurrent states, gate prefixes, triangular inverses, and sensitive accumulators in FP32. For query-key and key-key matrix products, FP16/BF16 inputs remain in their native dtype while `tl.dot` accumulates into FP32.

The first full-size state products used too much shared memory for some dimension combinations. I split the prediction and update products into tiles based on `INVERSE_BLOCK` instead of materializing one large product.

Validation and Performance

The public correctness suite passes for FP32, FP16, and BF16. I also checked the combined P9 state-passing case with `T=129`, `Dk=128`, `Dv=64`, a nonzero initial state, unequal channel gates, and a split after token 65. In FP16, the segmented output had maximum absolute error `1.221e-4` and relative RMS error `3.325e-4`; the segmented final state had maximum absolute error `5.398e-5` and relative RMS error `3.115e-4`.

The benchmark uses `B=1`, `H=4`, `T=1024`, and `Dk=Dv=64`. It includes input validation and output/state allocations, with five warm-up iterations followed by the average of twenty measured iterations.

Toolchain: Python 3.11.12, PyTorch 2.15.0+cu132, Triton 3.9.0, CUDA 13.2, and an NVIDIA Tesla T4 GPU.

GPU	Input dtype	Forward time (ms)
T4	FP32	4.183
T4	FP16	4.099
T4	BF16	3.462

Key Learnings

The most important lesson was how associativity turns linear attention into recurrent memory, while causality determines exactly which version of that memory a query may read. Chunking does not remove the sequential dependence of the boundary state, but it exposes parallel work in the output computation.

I also learned that numerical precision and GPU resource use need to be considered together. FP32 is valuable for recurrent and exponential calculations, but converting every matrix operand to FP32 can waste shared memory. Finally, the WY representation showed how a sequence of dependent DeltaNet corrections can be reorganized into chunk-level matrix operations without changing the recurrence.